

AI ASSIGNMENT- 1

NAME: SACHIN

ROLL.NO.: 251256

DEPT: B.SC (H.) CS

SUBJECT: ARTIFICIAL INTELLIGENCE (AI)

1. Write a PROLOG program to implement the family tree and demonstrate the family relationship.

```
P1-FamilyTree.pl X P2-ListConc.pl X P4-Sum.pl X P4-output.png X P3-ReversedList.pl X
Practicals > P1-FamilyTree.pl
1  parent(john, alice).
2  parent(john, bob).
3  parent(mary, alice).
4  parent(mary, bob).
5
6  parent(bob, charlie).
7  parent(susan, charlie).
8
9  parent(bob, david).
10 parent(susan, david).
11
12 % Father
13 father(X, Y) :-
14     parent(X, Y),
15     male(X).
16
17 % Mother
18 mother(X, Y) :-
19     parent(X, Y),
20     female(X).
21
22 male(john).
23 male(bob).
24 male(charlie).
25 male(david).
26
27 female(mary).
28 female(susan).
29 female(alice).
30
31 sibling(X, Y) :-
32     parent(P, X),
33     parent(P, Y),
34     X \= Y.
35
36 % Grandparent
37 grandparent(X, Y) :-
38     parent(X, Z),
39     parent(Z, Y).
40
41 % Grandfather
42 grandfather(X, Y) :-
43     grandparent(X, Y),
44     male(X).
45
46 % Grandmother
47 grandmother(X, Y) :-
48     grandparent(X, Y),
49     female(X).
50
51 % Uncle
52 uncle(X, Y) :-
53     male(X),
54     sibling(X, P),
55     parent(P, Y).
56
57 % Aunt
58 aunt(X, Y) :-
59     female(X),
60     sibling(X, P),
61     parent(P, Y).
```

```
PS D:\AI-Practicals-251256\Practical1> swipl
```

```
For online help and background, visit https://www.swi-prolog.org
```

```
For built-in help, use ?- help(Topic). or ?- apropos(Word).
```

```
1 ?- consult('D:/AI-Practicals-251256/Practical1/P1-FamilyTree.pl').  
true.
```

```
2 ?- parent(mary,alice).  
true.
```

```
3 ?- parent(john,bob).  
true.
```

```
4 ?- grandparent(mary,david).  
true.
```

```
5 ?- grandfather(john,charlie).  
true.
```

```
6 ?- grandmother(john,charlie).  
false.
```

```
7 ?- grandmother(mary,charlie).  
true.
```

```
8 ?- sibling(alice,bob).  
8 ?- sibling(alice,bob).  
Unknown action: (h for help)  
Action? ;  
true.
```

```
9 ?- uncle(john,susan).  
false.
```

```
10 ?- aunt(alice,david).  
true ;  
true.
```

```
11 ?- mother(susan,charlie).  
true.
```

2. Write a PROLOG program to implement `conc(L1, L2, L3)` where L2 is the list to be appended with L1 to get the resulted list L3.

```
P1-FamilyTree.pl X P2-ListConc.pl X P4-Sum.pl X P4-output.png X P3-ReversedList.pl X
Practicals > P2-ListConc.pl
1  conc([],L,L).
2  conc([H|T],L,[H|R]) :-
3  conc(T,L,R).
```

```
PS D:\AI-Practicals-251256> cd Practical
PS D:\AI-Practicals-251256\Practicals> swipl
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

1 ?- ['P2-ListConc.pl'].
true.

2 ?- conc([2,3,5],[6,3,5],R).
R = [2, 3, 5, 6, 3, 5].

3 ?- trace.
true.

[trace] 3 ?- conc([2,3,5],[6,3,5],R).
  Call: (12) conc([2, 3, 5], [6, 3, 5], _20076) ? creep
  Call: (13) conc([3, 5], [6, 3, 5], _21782) ? creep
  Call: (14) conc([5], [6, 3, 5], _22678) ? creep
  Call: (15) conc([], [6, 3, 5], _23574) ? creep
  Exit: (15) conc([], [6, 3, 5], [6, 3, 5]) ? creep
  Exit: (14) conc([5], [6, 3, 5], [5, 6, 3, 5]) ? creep
  Exit: (13) conc([3, 5], [6, 3, 5], [3, 5, 6, 3, 5]) ? creep
  Exit: (12) conc([2, 3, 5], [6, 3, 5], [2, 3, 5, 6, 3, 5]) ? creep
R = [2, 3, 5, 6, 3, 5].

[trace] 3 ?- █
```

3. Write a PROLOG program to implement reverse(L, R) where List L is original and List R is reversed list.

```
P1-FamilyTree.pl X P2-ListConc.pl X P4-Sum.pl X P4-output.png X P3-ReversedList.pl X
Practicals > P3-ReversedList.pl
1  reverse([], []).
2
3  reverse([H|T], R) :-
4      reverse(T, X),
5      append(X, [H], R).
```

```
PS D:\AI-Practicals-251256\Practicals> swipl
For built-in help, use ?- help(Topic). or ?- apropos(Word).

1 ?- ['P3-ReversedList.pl'].
true.

2 ?- reverse([1,2,3,4],R).
R = [4, 3, 2, 1].

3 ?- trace.
true.

[trace] 3 ?- reverse([1,2,3,4],R).
Call: (12) reverse([1, 2, 3, 4], _20068) ? creep
Call: (13) reverse([2, 3, 4], _22072) ? creep
Call: (14) reverse([3, 4], _22960) ? creep
Call: (15) reverse([4], _23848) ? creep
Call: (16) reverse([], _24736) ? creep
Exit: (16) reverse([], []) ? creep
Call: (16) lists:append([], [4], _23848) ? creep
Exit: (16) lists:append([], [4], [4]) ? creep
Exit: (15) reverse([4], [4]) ? creep
Call: (15) lists:append([4], [3], _22960) ? creep
Exit: (15) lists:append([4], [3], [4, 3]) ? creep
Exit: (14) reverse([3, 4], [4, 3]) ? creep
Call: (14) lists:append([4, 3], [2], _22072) ? creep
Exit: (14) lists:append([4, 3], [2], [4, 3, 2]) ? creep
Exit: (13) reverse([2, 3, 4], [4, 3, 2]) ? creep
Call: (13) lists:append([4, 3, 2], [1], _20068) ? creep
Exit: (13) lists:append([4, 3, 2], [1], [4, 3, 2, 1]) ? creep
Exit: (12) reverse([1, 2, 3, 4], [4, 3, 2, 1]) ? creep
R = [4, 3, 2, 1].
```

4. Write a PROLOG program to calculate the sum of two numbers.

```
P1-FamilyTree.pl X P2-ListConc.pl X P4-Sum.pl X P4-output.png X P3-ReversedList.pl X
Practicals > P4-Sum.pl
1 sum(A,B,C):-
2   C is A+B.
3
```

```
PS D:\AI-Practicals-251256> cd Practical
PS D:\AI-Practicals-251256\Practicals> swipl
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

1 ?- ['P3-Sum.pl'].
true.

2 ?- sum(60,54,C).
C = 114.

3 ?- trace.
true.

[trace] 3 ?- sum(60,54,C).
  Call: (12) sum(60, 54, _18832) ? creep
  Call: (13) _18832 is 60+54 ? creep
  Exit: (13) 114 is 60+54 ? creep
  Exit: (12) sum(60, 54, 114) ? creep
C = 114.
```

5. Write a PROLOG program to implement $\text{max}(X, Y, M)$ so that M is the maximum of two numbers X and Y .

```
P1-FamilyTree.pl X P2-ListConc.pl X P4-Sum.pl X P4-output.png X P3-ReversedList.pl X
Practicals > P5-Maxnum.pl
1 max(X,Y,M):-
2   (X >= Y -> M = X ; M = Y).
```

```
PS D:\AI-Practicals-251256\Practicals> swipl
```

For online help and background, visit <https://www.swi-prolog.org>

For built-in help, use `?- help(Topic).` or `?- apropos(Word).`

```
1 ?- ['P5-Maxnum.pl'].
true.
```

```
2 ?- max(7,4,M).
M = 7.
```

```
3 ?- max(54,78,M).
M = 78.
```

```
4 ?- trace.
true.
```

```
[trace] 4 ?- max(54,78,M).
  Call: (12) max(54, 78, _21164) ? creep
  Call: (13) 54>=78 ? creep
  Fail: (13) 54>=78 ? creep
  Redo: (12) max(54, 78, _21164) ? creep
  Call: (13) _21164=78 ? creep
  Exit: (13) 78=78 ? creep
  Exit: (12) max(54, 78, 78) ? creep
M = 78.
```

```
[trace] 4 ?- █
```

6. Write a program in PROLOG to implement factorial (N, F) where F represents the factorial of a number N.

```
P1-FamilyTree.pl X P2-ListConc.pl X P4-Sum.pl X P4-output.png X P3-ReversedList.pl X
Practicals > P6-Factorial.pl
1 factorial(0, 1).
2
3 factorial(N, F) :-
4     N > 0,
5     N1 is N - 1,
6     factorial(N1, F1),
7     F is N * F1.
```

```
1 ?- ['P6-Factorial.pl'].
true.

2 ?- factorial(7,F).
F = 5040 .

3 ?- trace.
true.

[trace] 3 ?- factorial(7,F).
Call: (12) factorial(7, _19952) ? creep
Exit: (13) 7>0 ? creep
Call: (13) _23340 is 7+ -1 ? creep
Exit: (13) 6 is 7+ -1 ? creep
Call: (13) factorial(6, _25114) ? creep
Call: (14) 6>0 ? creep
Exit: (14) 6>0 ? creep
Call: (14) _27780 is 6+ -1 ? creep
Exit: (14) 5 is 6+ -1 ? creep
Call: (14) factorial(5, _29554) ? creep
Call: (15) 5>0 ? creep
Exit: (15) 5>0 ? creep
Call: (15) _32220 is 5+ -1 ? creep
Exit: (15) 4 is 5+ -1 ? creep
Call: (15) factorial(4, _33994) ? creep
Call: (16) 4>0 ? creep
Exit: (16) 4>0 ? creep
Call: (16) _36660 is 4+ -1 ? creep
Exit: (16) 3 is 4+ -1 ? creep
Call: (16) factorial(3, _38434) ? creep
Call: (17) 3>0 ? creep
Exit: (17) 3>0 ? creep
Call: (17) _41100 is 3+ -1 ? creep
Call: (17) _41100 is 3+ -1 ? creep
Exit: (17) 2 is 3+ -1 ? creep
Call: (17) factorial(2, _42874) ? creep
Call: (18) 2>0 ? creep
Exit: (18) 2>0 ? creep
Call: (18) _45540 is 2+ -1 ? creep
Exit: (18) 1 is 2+ -1 ? creep
Call: (18) factorial(1, _47314) ? creep
Call: (19) 1>0 ? creep
Exit: (19) 1>0 ? creep
Call: (19) _49980 is 1+ -1 ? creep
Exit: (19) 0 is 1+ -1 ? creep
Call: (19) factorial(0, _51754) ? creep
Exit: (19) factorial(0, 1) ? creep
Call: (19) _47314 is 1*1 ? creep
Exit: (19) 1 is 1*1 ? creep
Exit: (18) factorial(1, 1) ? creep
Call: (18) _42874 is 2*1 ? creep
Exit: (18) 2 is 2*1 ? creep
Exit: (17) factorial(2, 2) ? creep
Call: (17) _38434 is 3*2 ? creep
Exit: (17) 6 is 3*2 ? creep
Exit: (16) factorial(3, 6) ? creep
Call: (16) _33994 is 4*6 ? creep
Exit: (16) 24 is 4*6 ? creep
Exit: (15) factorial(4, 24) ? creep
Call: (15) _126 is 5*24 ? creep
Exit: (15) 120 is 5*24 ? creep
Exit: (14) factorial(5, 120) ? creep
Call: (14) _122 is 6*120 ? creep
Exit: (14) 720 is 6*120 ? creep
Exit: (13) factorial(6, 720) ? creep
Call: (13) _62 is 7*720 ? creep
Exit: (13) 5040 is 7*720 ? creep
Exit: (12) factorial(7, 5040) ? creep
F = 5040 .
```

Thank You